

TUGAS BESAR IF2224

MILESTONE 4

Intermediate Code and Interpreter



Kelompok std_abs

Ega Luthfi Rais	13524115
Salman Faiz Assidqi	13524134
Ahmad Fauzan Putra	13524141
Leonardus Brain Fatolosja	13524146

DAFTAR ISI

DAFTAR ISI	2
BAB 1 LANDASAN TEORI	3
1.1 Decorated AST	3
1.2 Intermediate Code	3
1.3 Three-Address Code	4
1.4 Stack	5
1.5 Interpreter	6
BAB 2 PERANCANGAN & IMPLEMENTASI	7
2.1 Rancangan Intermediate Code Generator	7
2.2 Rancangan Interpreter	7
2.3 Alur Kerja Program	7
2.4 Implementasi Intermediate Code Generator	8
2.4.1 Instruction	8
2.4.2 Code Generator	8
2.5 Implementasi Interpreter	8
2.5.1 Stack Machine	8
2.5.2 Interpreter	8
2.5.3 Helper (runtimeValue.hpp)	8
2.6 Format Output	8
BAB 3 PENGUJIAN	9
3.1 Pengujian 1: input-1.txt	9
3.2 Pengujian 2: input-2.txt	10
3.3 Pengujian 3: input-3.txt	10
3.4 Pengujian 4: input-4.txt	12
3.5 Pengujian 5: input-5.txt	13
BAB 4 KESIMPULAN DAN SARAN	14
4.1 Kesimpulan	14
4.2 Saran	14
LAMPIRAN	15
REFERENSI	16

BAB 1 LANDASAN TEORI

1.1 Decorated AST

Abstract Syntax Tree adalah penyederhanaan parse tree yang membuang detail-detail sintaktis yang tidak berperan dalam analisis makna, sambil mempertahankan struktur logis program. Yang biasanya dihilangkan adalah Tanda baca pemisah: titik koma, koma, kurung, titik (kecuali untuk akses field), Keyword yang murni penanda struktur: THEN, DO, BEGIN, END, OF, dan Rantai nonterminal perantara yang hanya berfungsi mengkodekan presedensi operator. Yang biasanya dipertahankan adalah Operator sebagai label node internal (+, *, :=, if, while), Operand sebagai anak-anak node operator, dengan urutan dan kedalaman yang mencerminkan presedensi dan asosiativitas, dan Identifier dan literal sebagai node daun.

Decorated AST (atau Annotated AST) adalah AST yang setiap nodenya telah diperjelas dengan atribut-atribut hasil *semantic analysis*. Atribut yang dihasilkan meliputi:

- type, kode tipe hasil ekspresi (TYPE_INTEGER, TYPE_REAL, TYPE_BOOLEAN, TYPE_CHAR, TYPE_STRING, TYPE_ARRAY, TYPE_RECORD, TYPE_ENUM, TYPE_SUBRANGE, TYPE_VOID, atau TYPE_ERROR ketika type-checking gagal).
- tabIndex, referensi ke entri yang bersangkutan pada tabel identifier (tab); -1 jika node bukan referensi identifier.
- lev, lexical level di mana identifier tersebut dideklarasikan; level 0 adalah scope global tempat predefined identifier berada.
- isLValue, bernilai true jika node merepresentasikan addressable storage yang sah menjadi target assignment, dan false untuk literal serta hasil ekspresi sementara.

1.2 Intermediate Code

Intermediate Code adalah perantara antara source code dalam bahasa tingkat tinggi dengan kode yang akan dieksekusi oleh mesin (interpreter). Setelah proses lexical analysis, syntax analysis, dan semantic analysis selesai, program sudah valid secara sintaksis dan semantik, tetapi belum tentu efisien untuk langsung dieksekusi dari bentuk AST atau parse tree. Karena itu, dibutuhkan bentuk perantara yang lebih sederhana, linier, dan mudah diproses oleh tahap eksekusi berikutnya. Pada spesifikasi Arion, Intermediate Code dijelaskan sebagai hasil perataan struktur pohon yang kompleks menjadi daftar instruksi sekuensial yang sederhana, mirip bahasa mesin, tetapi masih cukup mudah dibaca manusia.

Peran utama Intermediate Code adalah memisahkan urusan front-end dan back-end compiler. Front-end bertugas memahami source code, sedangkan back-end bertugas mengeksekusi atau menerjemahkan representasi program tersebut. Dengan adanya Intermediate Code, compiler atau

interpreter tidak perlu langsung mengeksekusi AST yang bercabang-cabang, melainkan cukup membaca instruksi satu per satu.

Dalam konteks tugas Arion, Intermediate Code menjadi jembatan antara Decorated AST dan Interpreter. Decorated AST menyediakan informasi semantik seperti tipe data dan referensi tabel simbol, lalu informasi tersebut diterjemahkan menjadi instruksi operasional yang dapat dijalankan oleh interpreter. Dengan demikian, Intermediate Code bukan sekadar bentuk lain dari program, tetapi bentuk yang sudah dirancang agar mudah dieksekusi secara berurutan.

1.3 Three-Address Code

Three-Address Code (TAC) adalah salah satu bentuk intermediate representation dalam compiler yang digunakan untuk menyederhanakan ekspresi kompleks menjadi urutan instruksi sederhana. Setiap instruksi TAC umumnya hanya memuat maksimal tiga alamat, yaitu dua operand sumber dan satu tempat hasil. Bentuk dasarnya dapat berupa $a = b$, $a = op\ b$, atau $a = b\ op\ c$, dengan a , b , dan c dapat berupa nama variabel, konstanta, atau variabel sementara yang dibuat oleh compiler. Tujuan utama TAC adalah membuat urutan operasi menjadi eksplisit, sehingga lebih mudah dianalisis, dioptimasi, dan diterjemahkan menjadi kode mesin atau instruksi interpreter.

Dalam TAC, ekspresi yang panjang tidak dihitung sekaligus, tetapi dipecah menjadi beberapa operasi kecil menggunakan variabel sementara. Misalnya ekspresi $x := (a + b) * (c - d)$ dapat diterjemahkan menjadi $t1 := a + b$, $t2 := c - d$, $t3 := t1 * t2$, lalu $x := t3$. Dengan cara ini, struktur ekspresi yang awalnya berbentuk pohon dapat “diratakan” menjadi instruksi linier yang lebih dekat dengan cara kerja mesin eksekusi. TAC juga berguna untuk merepresentasikan alur kontrol seperti if, while, dan for menggunakan label serta instruksi lompat seperti goto atau conditional jump. TAC dipakai dalam optimasi kode, pembuatan kode, debugging, dan translasi bahasa karena bentuknya lebih sederhana daripada source code, tetapi masih lebih mudah dipahami daripada machine code.

TAC dapat direpresentasikan dalam beberapa bentuk, yaitu quadruple, triple, dan indirect triple. Quadruple menyimpan instruksi dalam empat field: operator, argumen pertama, argumen kedua, dan hasil. Bentuk ini mudah dipakai untuk optimasi karena hasil antara disimpan secara eksplisit dalam variabel sementara, tetapi dapat menghasilkan banyak temporary variable. Triple tidak menyimpan hasil dalam temporary variable eksplisit, melainkan mereferensikan hasil instruksi sebelumnya berdasarkan posisi instruksi tersebut. Bentuk ini lebih hemat temporary variable, tetapi lebih sulit diubah urutannya saat optimasi. Indirect triple menambahkan daftar pointer ke triple, sehingga perubahan urutan instruksi cukup dilakukan pada daftar pointer tanpa mengubah triple aslinya.

Proses pembentukan TAC biasanya dilakukan setelah tahap lexical analysis, syntax analysis, dan semantic analysis. Source code terlebih dahulu diubah menjadi token, lalu dibentuk menjadi

Abstract Syntax Tree, kemudian dicek secara semantik menggunakan informasi seperti tipe data dan symbol table. Setelah itu, compiler menelusuri AST dan menghasilkan instruksi TAC sesuai struktur ekspresi, assignment, percabangan, perulangan, dan pemanggilan prosedur. Untuk ekspresi aritmatika, compiler memecah operasi berdasarkan prioritas operator. Untuk ekspresi logika dan control flow, compiler menggunakan conditional jump dan label. Dengan demikian, TAC menjadi jembatan penting antara representasi program tingkat tinggi dan instruksi tingkat rendah yang siap dieksekusi atau diterjemahkan lebih lanjut.

1.4 Stack

Stack adalah struktur data yang bekerja dengan prinsip LIFO, yaitu Last In, First Out. Artinya, data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Dalam konteks interpreter, stack digunakan sebagai memori eksekusi untuk menyimpan nilai sementara, variabel lokal, argumen fungsi, return address, dan informasi lain yang diperlukan selama program berjalan.

Stack menjadi penting karena interpreter perlu tempat untuk menyimpan nilai saat instruksi dijalankan. Misalnya, instruksi LIT v dapat memasukkan literal v ke dalam stack, LOD a memuat nilai dari alamat tertentu, dan STO a menyimpan nilai ke alamat tertentu. Dalam spesifikasi Arion, interpreter menggunakan model stack machine, sehingga instruksi Intermediate Code dijalankan dengan bantuan stack penyimpanan.

Selain untuk penyimpanan nilai, stack juga berkaitan dengan scope dan pemanggilan fungsi. Ketika sebuah fungsi dipanggil, interpreter dapat membuat stack frame baru yang berisi variabel lokal, parameter, dan alamat kembali. Setelah fungsi selesai, stack frame tersebut dikeluarkan dari stack. Mekanisme ini membuat variabel lokal hanya hidup selama fungsi berjalan dan tidak mengganggu bagian program lain.

Namun, penggunaan stack juga harus dikontrol. Kesalahan seperti stack overflow, stack underflow, stack corruption, atau akses memori tidak valid dapat menyebabkan interpreter gagal menjalankan program dengan benar. Karena itu, implementasi stack dalam interpreter perlu memiliki batasan, validasi push/pop, serta penanganan error yang jelas.

1.5 Interpreter

Interpreter adalah program yang menjalankan instruksi dari bahasa tingkat tinggi dengan cara membaca, menerjemahkan, dan mengeksekusinya secara langsung, biasanya per baris atau per instruksi. Berbeda dari compiler yang menerjemahkan seluruh program menjadi kode mesin sebelum dijalankan, interpreter mengeksekusi program secara bertahap saat runtime. Karena itu, interpreter sering lebih mudah digunakan untuk debugging: ketika ada kesalahan, program dapat berhenti pada titik kesalahan tersebut sehingga programmer lebih mudah mengetahui bagian

mana yang perlu diperbaiki. Namun, konsekuensinya, program yang dijalankan dengan interpreter umumnya lebih lambat dibandingkan program hasil kompilasi penuh karena proses penerjemahan dan eksekusi terjadi berulang saat program berjalan.

Dalam sistem bahasa pemrograman, interpreter dapat bekerja langsung terhadap source code, tetapi pada banyak implementasi modern interpreter tidak selalu membaca teks mentah secara langsung. Source code biasanya melewati tahap awal seperti lexical analysis, parsing, dan semantic analysis, lalu diubah menjadi bentuk perantara seperti Abstract Syntax Tree, bytecode, atau intermediate code. Setelah itu, interpreter menjalankan representasi perantara tersebut. Pada konteks Arion, interpreter bertindak seperti virtual machine yang membaca Intermediate Code baris demi baris, mengatur instruction pointer, menyimpan nilai di memori, melakukan operasi aritmatika atau logika, menangani percabangan melalui jump/label, serta menghasilkan output saat menemukan instruksi seperti writeln.

Interpreter memiliki beberapa kelebihan. Pertama, proses pengembangan lebih fleksibel karena program dapat langsung dijalankan tanpa tahap kompilasi penuh. Kedua, error dapat dilaporkan lebih dekat dengan posisi eksekusi, sehingga cocok untuk pembelajaran, scripting, dan eksperimen cepat. Ketiga, interpreter lebih portabel jika program dijalankan di atas virtual machine yang sama, karena source code atau intermediate code tidak harus diubah menjadi kode mesin spesifik sejak awal. Namun, interpreter juga memiliki kelemahan, terutama dari sisi performa. Karena instruksi perlu dianalisis dan dijalankan saat runtime, eksekusinya bisa lebih lambat daripada program hasil compiler. Selain itu, untuk program interpreted, source code atau representasi perantaranya sering masih dibutuhkan ketika program dijalankan.

Secara arsitektural, interpreter biasanya memiliki siklus kerja seperti CPU sederhana, yaitu fetch, decode, dan execute. Pada tahap fetch, interpreter mengambil instruksi yang sedang ditunjuk oleh instruction pointer atau program counter. Pada tahap decode, interpreter memahami jenis instruksi tersebut, misalnya assignment, operasi aritmatika, input-output, atau jump. Pada tahap execute, interpreter menjalankan aksi yang sesuai, misalnya menghitung nilai, menyimpan variabel ke memori, memindahkan alur program, atau mencetak output. Siklus ini terus berulang sampai program selesai. Untuk mendukung eksekusi, interpreter juga membutuhkan struktur memori seperti stack, heap, tabel variabel, atau environment agar nilai dan scope program dapat dikelola dengan benar.

BAB 2 PERANCANGAN & IMPLEMENTASI

Modul ini terbagi ke dalam 2 tahap, yaitu Intermediate Code Generator dan Interpreter (untuk menjalankan program).

2.1 Rancangan Intermediate Code Generator

Intermediate Code Generator dirancang sebagai tahap konversi dari Decorated AST dan Symbol Table menjadi daftar instruksi linear. Modul ini akan diimplementasikan pada kelas *CodeGenerator*. Kelas akan menerima *ProgramNode* sebagai root AST hasil semantic analyzer dan *SymbolTable* yang sudah berisi informasi *tab*, *btab*, dan *atab*. Output dari proses ini adalah *CodeGenerationResult* yang berisi kumpulan dari instruction yang dihasilkan oleh intermediate code generator dalam bentuk *vector<Instruction>* dan daftar *CodeGenerationError*.

Instruksi intermediate code direpresentasikan oleh struct *Instruction*. Setiap instruksi memiliki Jenis instruksi (*InstructionCode*), Tingkat instruksi (*int level*), Argumen Instruksi (*InstructionArgument*), dan komentar. Argumen Instruksi dapat menyimpan alamat, nomor operasi, literal numerik, literal karakter, boolean, maupun string. Instruksi utama yang digunakan adalah:

- 'LIT': memasukkan literal ke operand stack.
- 'LOD': mengambil nilai dari alamat runtime tertentu.
- 'STO': menyimpan nilai top stack ke alamat runtime tertentu.
- 'CAL': memanggil prosedur/fungsi pada entry point tertentu.
- 'INT': mengalokasikan frame/memori untuk scope aktif.
- 'JMP': lompat tanpa syarat.
- 'JPC': lompat jika kondisi yang di-pop bernilai false.
- 'OPR': menjalankan operasi aritmetika, relasional, I/O, atau operasi built-in lain.
- 'RET': keluar dari frame prosedur/fungsi atau mengakhiri program.
- 'LDA', 'LDI', 'STI': ekstensi untuk alamat variabel, load indirect, dan store indirect, terutama untuk parameter 'var', array, dan record.

Generator dirancang menggunakan dua fase utama. Fase pertama adalah pengumpulan informasi entry point untuk setiap subprogram, baik prosedur maupun fungsi. Pada tahap ini, generator mencatat hubungan antara subprogram dan alamat awal eksekusinya di intermediate code. Dengan cara ini, pemanggilan subprogram tetap dapat diarahkan ke entry point yang benar meskipun instruksi untuk body subprogram belum sepenuhnya dibuat.

Fase kedua adalah pembuatan instruksi intermediate code. Generator terlebih dahulu menyiapkan instruksi lompat agar eksekusi program utama tidak langsung masuk ke body prosedur atau fungsi. Setelah seluruh instruksi subprogram selesai dibuat, target instruksi lompat tersebut diperbarui agar menunjuk ke awal program utama. Selanjutnya, program utama mengalokasikan ruang memori sesuai kebutuhan frame, membuat instruksi untuk statement utama, dan mengakhiri eksekusi dengan instruksi RET.

2.2 Rancangan Interpreter

Interpreter dirancang sebagai stack machine yang mengeksekusi intermediate code secara berurutan. Proses eksekusi dilakukan dengan siklus utama fetch, decode, dan execute. Pada tahap fetch, interpreter mengambil instruksi yang sedang ditunjuk oleh instruction pointer. Pada tahap decode, interpreter mengenali jenis instruksi yang harus dijalankan. Pada tahap execute, interpreter menjalankan aksi sesuai instruksi tersebut, misalnya memasukkan nilai ke stack, mengambil nilai dari memori, menyimpan nilai, melakukan operasi aritmatika, melakukan percabangan, memanggil subprogram, atau mengakhiri frame eksekusi.

Masukan interpreter berupa daftar instruksi intermediate code yang telah dihasilkan oleh generator. Selama eksekusi, interpreter menyimpan nilai sementara pada operand stack dan menyimpan variabel, parameter, serta informasi frame pada runtime stack. Hasil eksekusi berupa output program, terutama dari instruksi yang merepresentasikan write atau writeln, serta daftar error runtime apabila terjadi kesalahan seperti akses memori tidak valid, operand tidak sesuai, atau pembagian dengan nol. Eksekusi dimulai dari instruksi pertama dan terus berjalan sampai instruction pointer keluar dari rentang instruksi atau program mencapai instruksi terminasi.

Nilai yang diproses saat runtime dirancang memiliki informasi tipe agar interpreter dapat membedakan nilai integer, real, boolean, char, string, dan nilai kosong. Informasi tipe ini penting karena hasil operasi, penyimpanan variabel, dan pencetakan output harus mengikuti tipe nilai yang sedang dieksekusi. Stack machine menggunakan dua area penyimpanan utama. Area pertama adalah memori runtime yang menyimpan frame eksekusi, parameter, variabel lokal, dan informasi pemanggilan subprogram. Area kedua adalah operand stack yang digunakan untuk menyimpan nilai sementara saat evaluasi ekspresi, pengiriman argumen, dan pengambilan hasil operasi.

Setiap pemanggilan subprogram membentuk frame baru. Frame tersebut menyimpan informasi hubungan antar-scope dan alamat kembali, sehingga interpreter dapat kembali ke titik pemanggilan setelah prosedur atau fungsi selesai dieksekusi. Instruksi INT digunakan untuk menyiapkan ruang memori pada frame aktif, CAL digunakan untuk masuk ke frame subprogram baru, dan RET digunakan untuk keluar dari frame tersebut. Jika RET dijalankan pada program utama, eksekusi dianggap selesai.

Akses variabel dilakukan dengan menghitung alamat runtime berdasarkan lexical level dan offset variabel. Lexical level digunakan untuk menentukan frame mana yang harus diakses, terutama ketika terdapat subprogram bersarang. Setelah frame target ditemukan, offset variabel digunakan untuk membaca atau menulis nilai pada memori. Instruksi LOD digunakan untuk membaca nilai, sedangkan STO digunakan untuk menyimpan nilai. Untuk akses tidak langsung seperti parameter by-reference, array, atau record, interpreter menggunakan mekanisme alamat indirect melalui instruksi LDA, LDI, dan STI.

Instruksi OPR digunakan untuk menjalankan operasi bawaan. Operasi ini mencakup operasi aritmatika, operasi relasional, output, dan input. Operasi aritmatika memanfaatkan operand stack untuk mengambil operand dan menyimpan hasil. Operasi relasional menghasilkan nilai boolean yang kemudian dapat dipakai oleh instruksi percabangan seperti JPC. Instruksi output seperti write dan writeln mengambil nilai dari operand stack, mengubahnya ke bentuk teks, lalu menambahkannya ke output program.

Interpreter juga dirancang untuk mencatat error yang terjadi saat program dijalankan. Runtime error dapat muncul ketika terjadi pembagian dengan nol, penggunaan operand yang tidak sesuai dengan operasi, operand stack kosong saat nilai dibutuhkan, akses memori di luar batas yang valid, dan lain-lain. Jika kesalahan seperti ini terjadi, interpreter tidak hanya menghentikan atau mengganggu eksekusi, tetapi juga menyimpan informasi error agar dapat ditampilkan sebagai bagian dari hasil eksekusi program.

2.3 Alur Kerja Program

Eksekusi mode all (lexical + syntax + semantic + intermediate code + interpreter) berjalan sebagai berikut:

1. Source code dibaca lexer (milestone 1), menghasilkan vector<Token>.
2. Vector<Token> dikonsumsi parser recursive-descent (milestone 2), menghasilkan ParseNode root yang merepresentasikan parse tree.
3. `SemanticAnalyzer::analyze(parseTree, tokens)` dipanggil. Tahap konversi membangun ProgramNode (root AST) dari parse tree melalui rekursi `convertProgram` ke `convertDeclarationPart` ke `convertCompoundStatement`.
4. SymbolTable diinisialisasi di konstruktor: `btabs[0]` (blok global) dibuat dan diisi dengan predefined identifier.
5. `visit(astRoot)` dipanggil: traversal DFS men-decorate setiap node, mendaftarkan identifier ke `tab/btab/atab`, dan mereport semantic error ke `TypeChecker`.
6. `visitProgram()` mendaftarkan nama program pada symbol table pada lexical level 0, kemudian membuka scope program menggunakan `pushScope()`. Deklarasi dan body utama program kemudian diproses pada scope tersebut.

7. Setiap deklarasi variabel, konstanta, tipe, prosedur, dan fungsi diregistrasikan ke `tab`. Informasi blok prosedur, fungsi, atau record disimpan pada `btab`, sedangkan informasi array dan subrange disimpan pada `atab`.
8. Setiap penggunaan identifikasi diperiksa melalui lookup dari scope terdalam menuju scope terluar. Node AST kemudian diberi anotasi berupa tipe akhir, indeks symbol table, lexical level, informasi l-value, dan lokasi pada source code.
9. `TypeChecker` memeriksa kompatibilitas assignment, operator, kondisi kontrol, parameter pemanggilan subprogram, batas subrange, serta kesalahan deklarasi atau penggunaan identifikasi.
10. Terakhir, program akan mencetak token list, parse tree, decorated AST, simbol tabel (`tab`, `btab`, `atab`), dan daftar semantic error.
11. Jika tidak ada semantic error, `runCode(analyzer)` dipanggil untuk menjalankan intermediate code dan interpreter.
12. `runCode()` mengambil `ProgramNode ast` melalui `semanticAnalyzer.getAST()` dan `SymbolTable` melalui `semanticAnalyzer.getSymbolTable()`.
13. `CodeGenerator::generate(ast, symTab)` membuat intermediate code. Generator mengumpulkan entry point subprogram, men-generate body subprogram, mem-patch jump ke main program, mengalokasikan frame main dengan INT, men-generate statement utama, lalu menutup program dengan RET.
14. Jika code generation menghasilkan error, program akan berhenti dan daftar error disimpan pada `CodeGenerationResult.error`.
15. Jika code generation berhasil, `Interpreter::run(instructions)` dipanggil untuk mengeksekusi intermediate code.
16. Interpreter menjalankan instruksi dengan ketentuan sebagai berikut: LIT, LOD, STO, LDA, LDI, dan STI memanipulasi operand stack dan memori, JMP dan JPC mengatur control flow, CAL dan RET mengatur frame subprogram, dan OPR menjalankan operasi aritmetika, relasional, dan I/O.
17. Output dari `write/writeln` dikumpulkan pada `InterpreterResult.output`, sedangkan error runtime dikumpulkan pada `InterpreterResult.errors`.

2.4 Implementasi Intermediate Code Generator

Intermediate Code Generator diimplementasikan sebagai tahap setelah semantic analysis. Tahap ini menerima Decorated AST dan Symbol Table, kemudian menghasilkan daftar instruksi intermediate code yang dapat dieksekusi oleh interpreter. Implementasi utamanya berada pada folder `src/intermediate-code/`, yaitu pada file `instruction.hpp`, `codeGenerator.hpp`, dan `codeGenerator.cpp`.

2.4.1 Instruction

Instruksi intermediate code direpresentasikan oleh struct `Instruction`. Setiap instruksi memiliki empat komponen utama, yaitu:

1. `code`: jenis instruksi yang dijalankan.
2. `level`: selisih lexical level untuk akses variabel antar-scope.
3. `argument`: argumen instruksi, misalnya alamat, nilai literal, atau kode operasi.
4. `comment`: keterangan tambahan untuk mempermudah pembacaan output intermediate code.

Jenis instruksi didefinisikan dalam enum `InstructionCode`. Instruksi utama yang digunakan adalah `LIT`, `LOD`, `STO`, `CAL`, `INT`, `JMP`, `JPC`, `OPR`, dan `RET`. Selain itu, terdapat instruksi tambahan `LDA`, `LDI`, dan `STI` untuk mendukung akses alamat tidak langsung, seperti parameter by-reference, array, dan record.

Operasi yang dijalankan oleh instruksi `OPR` didefinisikan dalam enum `OperationCode`. Operasi ini mencakup operasi aritmatika seperti `ADD`, `SUB`, `MUL`, `DIV`, dan `MOD`; operasi relasional seperti `EQL`, `NEQ`, `LSS`, `LEQ`, `GTR`, dan `GEQ`; serta operasi output seperti `WRT` dan `WRTLN`.

Argumen instruksi menggunakan `std::variant<int, double, bool, char, string>`, sehingga satu instruksi dapat menyimpan berbagai jenis nilai, baik alamat integer, literal numerik, boolean, karakter, maupun string. File `instruction.hpp` juga menyediakan fungsi bantuan untuk mengubah `instruction code`, `operation code`, dan `argument` menjadi string agar intermediate code dapat dicetak dalam format yang mudah dibaca.

2.4.2 Code Generator

Code generator diimplementasikan dalam kelas `CodeGenerator`. Fungsi utamanya adalah `generate()`, yang menerima root AST bertipe `ProgramNode` dan `SymbolTable`. Fungsi ini menghasilkan `CodeGenerationResult`, yaitu hasil yang berisi daftar instruksi dan daftar error apabila terjadi masalah pada proses pembuatan kode.

Code generator bekerja dalam dua fase. Fase pertama adalah pengumpulan entry point subprogram melalui `collectSubprogramEntries()`. Pada tahap ini, prosedur dan fungsi yang terdapat pada program dicatat terlebih dahulu agar pemanggilan subprogram dapat mengetahui alamat tujuan eksekusinya.

Fase kedua adalah pembuatan instruksi. Proses ini dimulai dari `generateProgram()`. Generator membuat instruksi `JMP` di awal untuk melewati body prosedur dan fungsi, kemudian membuat instruksi untuk semua subprogram. Setelah itu, target `JMP` diperbarui agar mengarah ke awal program utama. Program utama kemudian mengalokasikan frame menggunakan `INT`, membuat instruksi untuk statement utama, dan mengakhiri eksekusi dengan `RET`.

Untuk subprogram, pembuatan kode dilakukan dengan mencatat entry point, mengalokasikan frame, menyimpan parameter ke slot memori, membuat body subprogram, lalu menutupnya dengan RET. Pada fungsi, nilai balik diambil dari variabel lokal yang memiliki nama sama dengan fungsi sebelum instruksi RET dijalankan.

Pembuatan statement dilakukan berdasarkan jenis node AST. Assignment menghasilkan instruksi untuk mengevaluasi ekspresi kanan lalu menyimpan hasilnya ke variabel tujuan. Statement if, while, repeat, dan for menggunakan kombinasi JMP dan JPC untuk membentuk control flow. Pemanggilan prosedur atau fungsi menggunakan CAL, sedangkan fungsi bawaan seperti write dan writeln langsung diterjemahkan menjadi operasi OPR.

Untuk ekspresi, code generator menggunakan pendekatan berbasis stack. Operand kiri dan kanan dibuat terlebih dahulu, kemudian operator diterjemahkan menjadi instruksi OPR. Literal diterjemahkan menjadi LIT, variabel diterjemahkan menjadi LOD, dan assignment ke variabel diterjemahkan menjadi STO. Untuk akses array, record, atau parameter by-reference, generator menghitung alamat variabel terlebih dahulu menggunakan instruksi LDA, lalu membaca atau menulis nilainya dengan LDI dan STI.

Dengan rancangan ini, intermediate code yang dihasilkan bersifat linear, tetapi tetap dapat merepresentasikan scope, pemanggilan subprogram, control flow, evaluasi ekspresi, dan operasi input/output.

2.5 Implementasi Interpreter

Interpreter diimplementasikan sebagai tahap eksekusi dari intermediate code yang telah dihasilkan oleh code generator. Interpreter membaca daftar instruksi secara berurutan, menjalankan operasi sesuai jenis instruksi, serta menghasilkan output runtime dari program. Implementasi interpreter berada pada folder src/interpreter/, yaitu stackMachine.hpp/cpp, interpreter.hpp/cpp, dan runtimeValue.hpp.

2.5.1 Stack Machine

Stack machine berfungsi sebagai penyimpanan runtime selama program dieksekusi. Komponen utamanya adalah memory, operands, dan informasi frame aktif. `memory` digunakan sebagai memori utama untuk menyimpan frame, parameter, variabel lokal, dan data runtime lain. `operands` digunakan sebagai operand stack untuk menyimpan nilai sementara saat evaluasi ekspresi, pemanggilan subprogram, dan hasil operasi.

Setiap pemanggilan subprogram membentuk frame baru. Frame menyimpan tiga informasi utama, yaitu static link, dynamic link, dan return address. Static link digunakan untuk mengakses

scope luar, dynamic link digunakan untuk kembali ke frame pemanggil, dan return address digunakan untuk mengetahui instruksi mana yang harus dijalankan setelah subprogram selesai.

Stack machine menyediakan operasi dasar seperti alokasi frame, push/pop operand, load/store memori, dan resolusi alamat berdasarkan lexical level. Resolusi alamat ini penting agar variabel pada scope luar tetap dapat diakses dari subprogram bersarang.

2.5.2 Interpreter

Interpreter menerima `vector<Instruction>` sebagai input. Eksekusi dimulai dari instruksi pertama dan dikendalikan oleh instruction pointer. Pada setiap langkah, interpreter mengambil instruksi aktif, menentukan jenis instruksi, lalu menjalankan aksi yang sesuai.

Instruksi LIT memasukkan literal ke operand stack. LOD mengambil nilai dari memori, sedangkan STO menyimpan nilai ke memori. INT mengalokasikan ruang frame. CAL membuat frame baru dan memindahkan eksekusi ke subprogram. RET keluar dari frame aktif. JMP dan JPC digunakan untuk percabangan dan loop. OPR digunakan untuk operasi aritmetika, relasional, input, dan output.

Interpreter juga mendukung akses alamat tidak langsung melalui LDA, LDI, dan STI. Instruksi ini digunakan untuk kasus seperti parameter by-reference, array, dan record. LDA mengambil alamat variabel, LDI membaca nilai dari alamat tersebut, dan STI menyimpan nilai ke alamat yang dihitung.

Hasil eksekusi disimpan dalam `InterpreterResult`. Struktur ini memuat output program, daftar runtime error, jumlah instruksi yang dieksekusi, dan informasi input untuk operasi read/readln. Runtime error dapat muncul dari pembagian dengan nol, operand stack kosong, tipe operand yang tidak sesuai, atau akses memori yang tidak valid.

2.5.3 Helper (runtimeValue.hpp)

File `runtimeValue.hpp` berisi representasi nilai yang digunakan selama eksekusi program. Nilai runtime direpresentasikan oleh `RuntimeValue`, yang menyimpan tipe nilai dan isi nilainya.

Tipe runtime didefinisikan dalam `RuntimeType`, yaitu VOID, INTEGER, REAL, BOOLEAN, CHAR, dan STRING. Isi nilai disimpan menggunakan `std::variant<int, double, bool, char, string>`, sehingga satu struktur dapat mewakili berbagai jenis data. Helper ini juga menyediakan fungsi untuk mengubah tipe runtime menjadi string dan mengubah nilai runtime menjadi teks output. Fungsi tersebut digunakan terutama saat menjalankan instruksi output seperti `write` dan `writeln`.

Selain itu, terdapat helper untuk menentukan apakah suatu nilai dianggap false saat digunakan oleh instruksi percabangan bersyarat. Nilai boolean false, integer 0, dan real 0.0 dianggap sebagai false. Ini digunakan oleh instruksi JPC untuk menentukan apakah interpreter harus melakukan jump atau melanjutkan eksekusi.

2.6 Format Output

Output Milestone 4 terdiri dari dua bagian utama, yaitu intermediate code dan runtime output. Intermediate code menunjukkan hasil translasi dari Decorated AST menjadi instruksi yang dapat dieksekusi interpreter, sedangkan runtime output menunjukkan hasil eksekusi program, terutama keluaran dari write dan writeln.

Format intermediate code dicetak satu instruksi per baris dengan bentuk umum:

```
<nomor_baris> <instruction_code> <level> <argument> ; <comment>
```

Keterangan setiap bagian:

1. nomor_baris: nomor urut instruksi pada intermediate code.
2. instruction_code: jenis instruksi, misalnya LIT, LOD, STO, CAL, INT, JMP, JPC, OPR, atau RET.
3. level: selisih lexical level yang digunakan terutama untuk akses variabel antar-scope.
4. argument: argumen instruksi, misalnya nilai literal, alamat variabel, target jump, entry point subprogram, atau nomor operasi.
5. comment: keterangan tambahan agar instruksi lebih mudah dipahami.

Runtime output dicetak setelah intermediate code. Bagian ini berisi teks yang benar-benar dihasilkan oleh program ketika dijalankan. Output berasal dari instruksi OPR WRT dan OPR WRTLN, yang merepresentasikan write dan writeln pada source code.

Jika program tidak memiliki write atau writeln, maka bagian runtime output dapat kosong. Jika terjadi runtime error, error disimpan sebagai hasil eksekusi dan dapat ditampilkan sebagai daftar error runtime, misalnya pembagian dengan nol, akses memori tidak valid, atau operand stack kosong.

BAB 3 PENGUJIAN

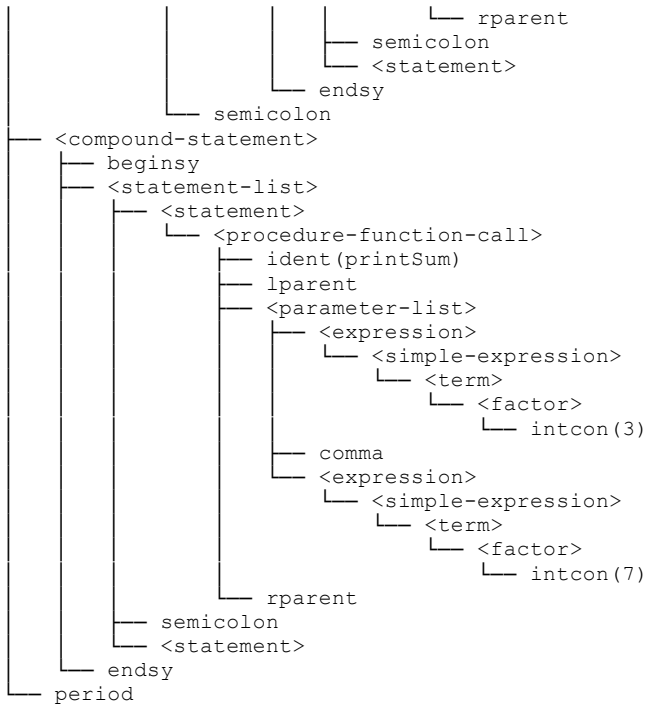
3.1 Pengujian 1: input-7.txt

Masukan (input)
<pre>program ProcParamTest; var a, b: integer; procedure printSum(x: integer; y: integer); begin writeln(x + y); end; begin printSum(3, 7); end.</pre>
Keluaran (output)
<pre>programsy ident(ProcParamTest) semicolon varsy ident(a) comma ident(b) colon ident(integer) semicolon proceduresy ident(printSum) lparent ident(x) colon ident(integer) semicolon ident(y) colon ident(integer) rparent semicolon beginsy ident(writeln) lparent ident(x) plus ident(y) rparent</pre>

```

semicolon
endsy
semicolon
beginsy
ident (printSum)
lparent
intcon(3)
comma
intcon(7)
rparent
semicolon
endsy
period
<Program>
├── <Program-header>
│   ├── programsy
│   ├── ident (ProcParamTest)
│   └── semicolon
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── varsy
│   │   ├── <identifier-list>
│   │   │   ├── ident (a)
│   │   │   ├── comma
│   │   │   └── ident (b)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (integer)
│   │   └── semicolon
│   └── <subprogram-declaration>
│       ├── <procedure-declaration>
│       │   ├── proceduresy
│       │   ├── ident (printSum)
│       │   ├── <formal-parameter-list>
│       │   │   ├── lparent
│       │   │   ├── <parameter-group>
│       │   │   │   ├── <identifier-list>
│       │   │   │   │   └── ident (x)
│       │   │   │   ├── colon
│       │   │   │   └── ident (integer)
│       │   │   └── semicolon
│       │   │   ├── <parameter-group>
│       │   │   │   ├── <identifier-list>
│       │   │   │   │   └── ident (y)
│       │   │   │   ├── colon
│       │   │   │   └── ident (integer)
│       │   │   └── rparent
│       │   └── semicolon
│       └── <Block>
│           ├── <declaration-part>
│           └── <compound-statement>
│               ├── beginsy
│               └── <statement-list>
│                   ├── <statement>
│                   │   ├── <procedure-function-call>
│                   │   │   ├── ident (writeln)
│                   │   │   ├── lparent
│                   │   │   ├── <parameter-list>
│                   │   │   │   └── <expression>
│                   │   │   │       ├── <simple-expression>
│                   │   │   │       │   ├── <term>
│                   │   │   │       │   │   ├── <factor>
│                   │   │   │       │   │   │   └── <variable>
│                   │   │   │       │   │   │       └── ident (x)
│                   │   │   │       │   ├── <additive-operator>
│                   │   │   │       │   │   └── plus
│                   │   │   │       │   └── <term>
│                   │   │   │       │       ├── <factor>
│                   │   │   │       │       │   └── <variable>
│                   │   │   │       │       │       └── ident (y)

```



===== Decorated AST =====

```

Program: ProcParamTest [tab=43] type=void lev=0 line=1:1
| Declarations: [3 declarations] line=2:1
| | VarDecl: a [tab=44] type=integer lev=1 line=3:9 <type simple=integer
| | resolved=integer>
| | VarDecl: b [tab=45] type=integer lev=1 line=3:9 <type simple=integer
| | resolved=integer>
| | Procedure: printSum [tab=46] type=void lev=1 line=5:1
| | | Params:
| | | | VarDecl: x [tab=47] type=integer lev=2 line=5:20 <type simple=integer
| | | | resolved=integer>
| | | | VarDecl: y [tab=48] type=integer lev=2 line=5:32 <type simple=integer
| | | | resolved=integer>
| | | | Locals:
| | | | Block: [2 statements] line=7:3
| | | | | Call: writeln [tab=39] type=void lev=0 args=1 line=7:3
| | | | | | BinOp: + type=integer line=7:13
| | | | | | | Var: x [tab=47] type=integer lev=2 lval=true line=7:11
| | | | | | | Var: y [tab=48] type=integer lev=2 lval=true line=7:15
| | | | | EmptyStmt
| | | Block: [2 statements] line=11:3
| | | | Call: printSum [tab=46] type=void lev=1 args=2 line=11:3
| | | | | Number: 3 type=integer line=11:12
| | | | | Number: 7 type=integer line=11:15
| | | | EmptyStmt

```

=== Symbol Table (tab) ===

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	keyword	void	0	value	0	0	-
1	array	keyword	void	0	value	0	0	-
2	begin	keyword	void	0	value	0	0	-
3	case	keyword	void	0	value	0	0	-
4	const	keyword	void	0	value	0	0	-
5	div	keyword	void	0	value	0	0	-
6	downto	keyword	void	0	value	0	0	-
7	do	keyword	void	0	value	0	0	-
8	else	keyword	void	0	value	0	0	-
9	end	keyword	void	0	value	0	0	-
10	for	keyword	void	0	value	0	0	-

11	function	keyword	void	0	value	0	0	-
12	if	keyword	void	0	value	0	0	-
13	mod	keyword	void	0	value	0	0	-
14	not	keyword	void	0	value	0	0	-
15	of	keyword	void	0	value	0	0	-
16	or	keyword	void	0	value	0	0	-
17	procedure	keyword	void	0	value	0	0	-
18	program	keyword	void	0	value	0	0	-
19	record	keyword	void	0	value	0	0	-
20	repeat	keyword	void	0	value	0	0	-
21	integer	keyword	void	0	value	0	0	-
22	real	keyword	void	0	value	0	0	-
23	boolean	keyword	void	0	value	0	0	-
24	char	keyword	void	0	value	0	0	-
25	string	keyword	void	0	value	0	0	-
26	then	keyword	void	0	value	0	0	-
27	to	keyword	void	0	value	0	0	-
28	type	keyword	void	0	value	0	0	-
29	until	keyword	void	0	value	0	0	-
30	var	keyword	void	0	value	0	0	-
31	while	keyword	void	0	value	0	0	-
32	integer	type	integer	0	value	0	1	-
33	real	type	real	0	value	0	1	32
34	boolean	type	boolean	0	value	0	1	33
35	char	type	char	0	value	0	1	34
36	string	type	string	0	value	0	1	35
37	true	constant	boolean	0	value	0	1	36
38	false	constant	boolean	0	value	0	0	37
39	writeln	procedure	void	0	value	0	0	38
40	readln	procedure	void	0	value	0	0	39
41	write	procedure	void	0	value	0	0	40
42	read	procedure	void	0	value	0	0	41
43	ProcParamTest	program	void	1	value	0	0	42
44	a	variable	integer	0	value	1	0	-
45	b	variable	integer	0	value	1	1	44
46	printSum	procedure	void	2	value	1	0	45
47	x	variable	integer	0	value	2	0	-
48	y	variable	integer	0	value	2	1	47

=== Block Table (btab) ===

idx	last	lpar	psze	vsze
0	43	0	0	0
1	46	-	0	2
2	48	48	2	0

=== Array Table (atab) ===

idx	xtyp	etyp	eref	low	high	elsz	size
(empty)							

===== Intermediate Code =====

```

1 JMP 0 9 ; skip subprogram bodies
2 INT 0 3 ; printSum frame (3 slots)
3 STO 0 4 ; param y
4 STO 0 3 ; param x
5 LOD 0 3 ; load x
6 LOD 0 4 ; load y
7 OPR 0 2 ; op +
8 OPR 0 14 ; writeln arg
9 RET 0 0 ; return from printSum
10 INT 0 5 ; main program frame (5 slots)
11 LIT 0 3 ; int 3
12 LIT 0 7 ; int 7
13 CAL 0 1 ; call printSum
14 RET 0 0 ; program end

```

===== Runtime Output =====

10

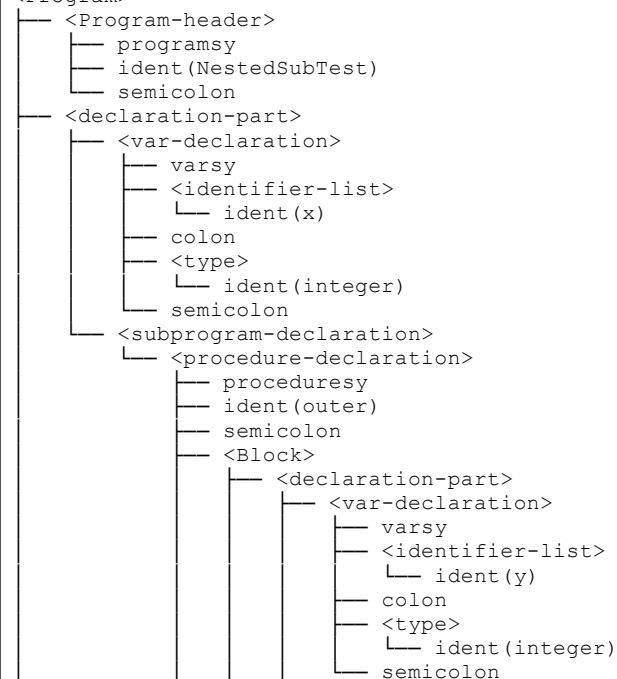
3.2 Pengujian 2: input-9.txt

Masukan (input)
<pre>program NestedSubTest; var x: integer; procedure outer; var y: integer; procedure inner; begin y := y + 10; end; begin y := 5; inner(); writeln('y from inner = ', y); end; begin outer(); end.</pre>
Keluaran (output)
<pre>programsy ident(NestedSubTest) semicolon varsy ident(x) colon ident(integer) semicolon proceduresy ident(outer) semicolon varsy</pre>

```

ident(y)
colon
ident(integer)
semicolon
proceduresy
ident(inner)
semicolon
beginsy
ident(y)
becomes
ident(y)
plus
intcon(10)
semicolon
endsy
semicolon
beginsy
ident(y)
becomes
intcon(5)
semicolon
ident(inner)
lparent
rparent
semicolon
ident(writeln)
lparent
string('y from inner = ')
comma
ident(y)
rparent
semicolon
endsy
semicolon
beginsy
ident(outer)
lparent
rparent
semicolon
endsy
period
<Program>

```




```

      ┌── endsy
      └── semicolon
    <compound-statement>
      ┌── beginsy
      └── <statement-list>
          ┌── <statement>
              └── <procedure-function-call>
                  ├── ident(outer)
                  ├── lparent
                  └── rparent
          └── semicolon
      <statement>
    endsy
  period

```

===== Decorated AST =====

```

Program: NestedSubTest [tab=43] type=void lev=0 line=1:1
├── Declarations: [2 declarations] line=2:1
│   ├── VarDecl: x [tab=44] type=integer lev=1 line=3:6 <type simple=integer
│   │   resolved=integer>
│   └── Procedure: outer [tab=45] type=void lev=1 line=5:1
│       ├── Locals:
│       │   └── VarDecl: y [tab=46] type=integer lev=2 line=7:6 <type simple=integer
│       │       resolved=integer>
│       └── Procedure: inner [tab=47] type=void lev=2 line=9:3
│           ├── Locals:
│           │   └── Block: [2 statements] line=11:5
│           │       ├── Assign: type=integer line=11:5
│           │       │   ├── Var: y [tab=46] type=integer lev=2 lval=true line=11:5
│           │       │   └── BinOp: + type=integer line=11:12
│           │       │       ├── Var: y [tab=46] type=integer lev=2 lval=true line=11:10
│           │       │       └── Number: 10 type=integer line=11:14
│           │       └── EmptyStmt
│           └── Block: [4 statements] line=15:3
│               ├── Assign: type=integer line=15:3
│               │   ├── Var: y [tab=46] type=integer lev=2 lval=true line=15:3
│               │   └── Number: 5 type=integer line=15:8
│               ├── Call: inner [tab=47] type=void lev=2 args=0 line=16:3
│               ├── Call: writeln [tab=39] type=void lev=0 args=2 line=17:3
│               │   ├── String: "y from inner = " type=string length=15 line=17:11
│               │   └── Var: y [tab=46] type=integer lev=2 lval=true line=17:30
│               └── EmptyStmt
└── Block: [2 statements] line=21:3
    ├── Call: outer [tab=45] type=void lev=1 args=0 line=21:3
    └── EmptyStmt

```

=== Symbol Table (tab) ===

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	keyword	void	0	value	0	0	-
1	array	keyword	void	0	value	0	0	-
2	begin	keyword	void	0	value	0	0	-
3	case	keyword	void	0	value	0	0	-
4	const	keyword	void	0	value	0	0	-
5	div	keyword	void	0	value	0	0	-
6	downto	keyword	void	0	value	0	0	-
7	do	keyword	void	0	value	0	0	-
8	else	keyword	void	0	value	0	0	-
9	end	keyword	void	0	value	0	0	-
10	for	keyword	void	0	value	0	0	-
11	function	keyword	void	0	value	0	0	-
12	if	keyword	void	0	value	0	0	-
13	mod	keyword	void	0	value	0	0	-
14	not	keyword	void	0	value	0	0	-
15	of	keyword	void	0	value	0	0	-
16	or	keyword	void	0	value	0	0	-
17	procedure	keyword	void	0	value	0	0	-
18	program	keyword	void	0	value	0	0	-
19	record	keyword	void	0	value	0	0	-
20	repeat	keyword	void	0	value	0	0	-

21	integer	keyword	void	0	value	0	0	-
22	real	keyword	void	0	value	0	0	-
23	boolean	keyword	void	0	value	0	0	-
24	char	keyword	void	0	value	0	0	-
25	string	keyword	void	0	value	0	0	-
26	then	keyword	void	0	value	0	0	-
27	to	keyword	void	0	value	0	0	-
28	type	keyword	void	0	value	0	0	-
29	until	keyword	void	0	value	0	0	-
30	var	keyword	void	0	value	0	0	-
31	while	keyword	void	0	value	0	0	-
32	integer	type	integer	0	value	0	1	-
33	real	type	real	0	value	0	1	32
34	boolean	type	boolean	0	value	0	1	33
35	char	type	char	0	value	0	1	34
36	string	type	string	0	value	0	1	35
37	true	constant	boolean	0	value	0	1	36
38	false	constant	boolean	0	value	0	0	37
39	writeln	procedure	void	0	value	0	0	38
40	readln	procedure	void	0	value	0	0	39
41	write	procedure	void	0	value	0	0	40
42	read	procedure	void	0	value	0	0	41
43	NestedSubTest	program	void	1	value	0	0	42
44	x	variable	integer	0	value	1	0	-
45	outer	procedure	void	2	value	1	0	44
46	y	variable	integer	0	value	2	0	-
47	inner	procedure	void	3	value	2	0	46

=== Block Table (btab) ===

idx	last	lpar	psze	vsze
0	43	0	0	0
1	45	-	0	1
2	47	-	0	1
3	-	-	0	0

=== Array Table (atab) ===

idx	xtyp	etyp	eref	low	high	elsz	size
(empty)							

===== Intermediate Code =====

```

1 JMP 0 17 ; skip subprogram bodies
2 INT 0 4 ; outer frame (4 slots)
3 JMP 0 9 ; skip nested subprograms
4 INT 0 3 ; inner frame (3 slots)
5 LOD 1 3 ; load y
6 LIT 0 10 ; int 10
7 OPR 0 2 ; op +
8 STO 1 3 ; store y
9 RET 0 0 ; return from inner
10 LIT 0 5 ; int 5
11 STO 0 3 ; store y
12 CAL 0 3 ; call inner
13 LOD 0 3 ; load y
14 LIT 0 y from inner = ; string "y from inner = "
15 OPR 0 13 ; write arg
16 OPR 0 14 ; writeln arg
17 RET 0 0 ; return from outer
18 INT 0 4 ; main program frame (4 slots)
19 CAL 0 1 ; call outer
20 RET 0 0 ; program end

```

===== Runtime Output =====

y from inner = 15

3.3 Pengujian 3: input-10.txt

Masukan (input)

```
program FuncComposeTest;
var
  x: integer;

function square(n: integer): integer;
begin
  square := n * n;
end;

function doubleValue(n: integer): integer;
begin
  doubleValue := n * 2;
end;

begin
  x := 3;
  writeln('square(3) = ', square(x));
  writeln('double(9) = ', doubleValue(9));
  writeln('compose: ', doubleValue(square(x)));
end.
```

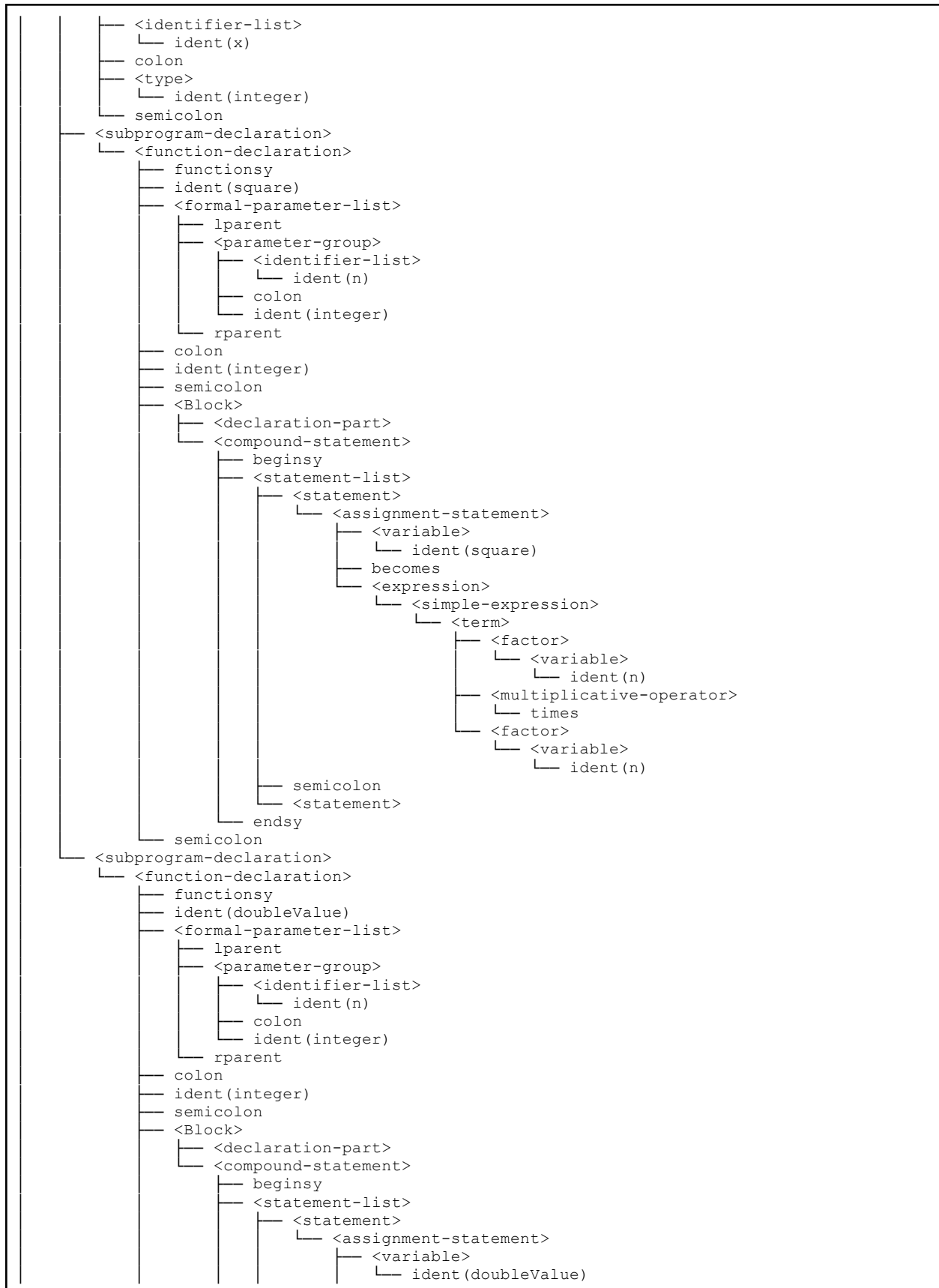
Keluaran (output)

```
programsy
ident(FuncComposeTest)
semicolon
varsy
ident(x)
colon
ident(integer)
semicolon
functionsy
ident(square)
lparent
ident(n)
colon
ident(integer)
rparent
colon
ident(integer)
semicolon
beginsy
ident(square)
becomes
ident(n)
times
ident(n)
```

```

semicolon
endsy
semicolon
functionsy
ident(doubleValue)
lparent
ident(n)
colon
ident(integer)
rparent
colon
ident(integer)
semicolon
beginsy
ident(doubleValue)
becomes
ident(n)
times
intcon(2)
semicolon
endsy
semicolon
beginsy
ident(x)
becomes
intcon(3)
semicolon
ident(writeln)
lparent
string('square(3) = ')
comma
ident(square)
lparent
ident(x)
rparent
rparent
semicolon
ident(writeln)
lparent
string('double(9) = ')
comma
ident(doubleValue)
lparent
intcon(9)
rparent
rparent
semicolon
ident(writeln)
lparent
string('compose: ')
comma
ident(doubleValue)
lparent
ident(square)
lparent
ident(x)
rparent
rparent
rparent
semicolon
endsy
period
<Program>
├─ <Program-header>
│   ├── programsy
│   └── ident(FuncComposeTest)
│       └── semicolon
├─ <declaration-part>
│   └── <var-declaration>
│       └── varsy

```






```

    └─ Assign: type=integer line=7:3
        └─ Var: square [tab=47] type=integer lev=2 lval=true line=7:3
        └─ BinOp: * type=integer line=7:15
            └─ Var: n [tab=46] type=integer lev=2 lval=true line=7:13
            └─ Var: n [tab=46] type=integer lev=2 lval=true line=7:17
        └─ EmptyStmt
    └─ Function: doubleValue [tab=48] type=void lev=1 line=10:1 returns=integer <type
simple=integer resolved=integer>
    └─ Params:
        └─ VarDecl: n [tab=49] type=integer lev=2 line=10:22 <type simple=integer
resolved=integer>
    └─ Locals:
        └─ Block: [2 statements] line=12:3
            └─ Assign: type=integer line=12:3
                └─ Var: doubleValue [tab=50] type=integer lev=2 lval=true line=12:3
                └─ BinOp: * type=integer line=12:20
                    └─ Var: n [tab=49] type=integer lev=2 lval=true line=12:18
                    └─ Number: 2 type=integer line=12:22
            └─ EmptyStmt
    └─ Block: [5 statements] line=16:3
        └─ Assign: type=integer line=16:3
            └─ Var: x [tab=44] type=integer lev=1 lval=true line=16:3
            └─ Number: 3 type=integer line=16:8
        └─ Call: writeln [tab=39] type=void lev=0 args=2 line=17:3
            └─ String: "square(3) =" type=string length=12 line=17:11
            └─ Call: square [tab=45] type=integer lev=1 args=1 line=17:27
                └─ Var: x [tab=44] type=integer lev=1 lval=true line=17:34
        └─ Call: writeln [tab=39] type=void lev=0 args=2 line=18:3
            └─ String: "double(9) =" type=string length=12 line=18:11
            └─ Call: doubleValue [tab=48] type=integer lev=1 args=1 line=18:27
                └─ Number: 9 type=integer line=18:39
        └─ Call: writeln [tab=39] type=void lev=0 args=2 line=19:3
            └─ String: "compose: " type=string length=9 line=19:11
            └─ Call: doubleValue [tab=48] type=integer lev=1 args=1 line=19:24
                └─ Call: square [tab=45] type=integer lev=1 args=1 line=19:36
                    └─ Var: x [tab=44] type=integer lev=1 lval=true line=19:43
        └─ EmptyStmt

```

=== Symbol Table (tab) ===

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	keyword	void	0	value	0	0	-
1	array	keyword	void	0	value	0	0	-
2	begin	keyword	void	0	value	0	0	-
3	case	keyword	void	0	value	0	0	-
4	const	keyword	void	0	value	0	0	-
5	div	keyword	void	0	value	0	0	-
6	downto	keyword	void	0	value	0	0	-
7	do	keyword	void	0	value	0	0	-
8	else	keyword	void	0	value	0	0	-
9	end	keyword	void	0	value	0	0	-
10	for	keyword	void	0	value	0	0	-
11	function	keyword	void	0	value	0	0	-
12	if	keyword	void	0	value	0	0	-
13	mod	keyword	void	0	value	0	0	-
14	not	keyword	void	0	value	0	0	-
15	of	keyword	void	0	value	0	0	-
16	or	keyword	void	0	value	0	0	-
17	procedure	keyword	void	0	value	0	0	-
18	program	keyword	void	0	value	0	0	-
19	record	keyword	void	0	value	0	0	-
20	repeat	keyword	void	0	value	0	0	-
21	integer	keyword	void	0	value	0	0	-
22	real	keyword	void	0	value	0	0	-
23	boolean	keyword	void	0	value	0	0	-
24	char	keyword	void	0	value	0	0	-
25	string	keyword	void	0	value	0	0	-
26	then	keyword	void	0	value	0	0	-
27	to	keyword	void	0	value	0	0	-
28	type	keyword	void	0	value	0	0	-

29	until	keyword	void	0	value	0	0	-
30	var	keyword	void	0	value	0	0	-
31	while	keyword	void	0	value	0	0	-
32	integer	type	integer	0	value	0	1	-
33	real	type	real	0	value	0	1	32
34	boolean	type	boolean	0	value	0	1	33
35	char	type	char	0	value	0	1	34
36	string	type	string	0	value	0	1	35
37	true	constant	boolean	0	value	0	1	36
38	false	constant	boolean	0	value	0	0	37
39	writeln	procedure	void	0	value	0	0	38
40	readln	procedure	void	0	value	0	0	39
41	write	procedure	void	0	value	0	0	40
42	read	procedure	void	0	value	0	0	41
43	FuncComposeTestprogram		void	1	value	0	0	42
44	x	variable	integer	0	value	1	0	-
45	square	function	integer	2	value	1	0	44
46	n	variable	integer	0	value	2	0	-
47	square	variable	integer	0	value	2	1	46
48	doubleValue	function	integer	3	value	1	0	45
49	n	variable	integer	0	value	2	0	-
50	doubleValue	variable	integer	0	value	2	1	49

=== Block Table (btab) ===

idx	last	lpar	psze	vsze
0	43	0	0	0
1	48	-	0	1
2	47	46	1	1
3	50	49	1	1

=== Array Table (atab) ===

idx	xtp	etyp	eref	low	high	elsz	size
(empty)							

===== Intermediate Code =====

```

1 JMP 0 17 ; skip subprogram bodies
2 INT 0 4 ; square frame (4 slots)
3 STO 0 3 ; param n
4 LOD 0 3 ; load n
5 LOD 0 3 ; load n
6 OPR 0 4 ; op *
7 STO 0 4 ; store square
8 LOD 0 4 ; load square return value
9 RET 0 1 ; return from square
10 INT 0 4 ; doubleValue frame (4 slots)
11 STO 0 3 ; param n
12 LOD 0 3 ; load n
13 LIT 0 2 ; int 2
14 OPR 0 4 ; op *
15 STO 0 4 ; store doubleValue
16 LOD 0 4 ; load doubleValue return value
17 RET 0 1 ; return from doubleValue
18 INT 0 4 ; main program frame (4 slots)
19 LIT 0 3 ; int 3
20 STO 0 3 ; store x
21 LOD 0 3 ; load x
22 CAL 0 1 ; call square
23 LIT 0 square(3) = ; string "square(3) = "
24 OPR 0 13 ; write arg
25 OPR 0 14 ; writeln arg
26 LIT 0 9 ; int 9
27 CAL 0 9 ; call doubleValue
28 LIT 0 double(9) = ; string "double(9) = "
29 OPR 0 13 ; write arg
30 OPR 0 14 ; writeln arg
31 LOD 0 3 ; load x
32 CAL 0 1 ; call square
33 CAL 0 9 ; call doubleValue

```

```
34 LIT 0 compose: ; string "compose: "  
35 OPR 0 13 ; write arg  
36 OPR 0 14 ; writeln arg  
37 RET 0 0 ; program end
```

===== Runtime Output =====

```
square(3) = 9  
double(9) = 18  
compose: 18
```

3.4 Pengujian 4: input-13.txt

Masukan (input)
<pre>program testNestedVarParam; var result: integer; procedure outer(var x: integer); var temp: integer; procedure inner; begin x := x * 2 end; begin x := x + 5; inner end; begin result := 10; writeln(result); outer(result); writeln(result) end.</pre>
Keluaran (output)
<pre>programsy ident(testNestedVarParam) semicolon varsy ident(result) colon ident(integer) semicolon proceduresy ident(outer) lparent varsy ident(x) colon ident(integer) rparent semicolon varsy ident(temp) colon ident(integer)</pre>

```

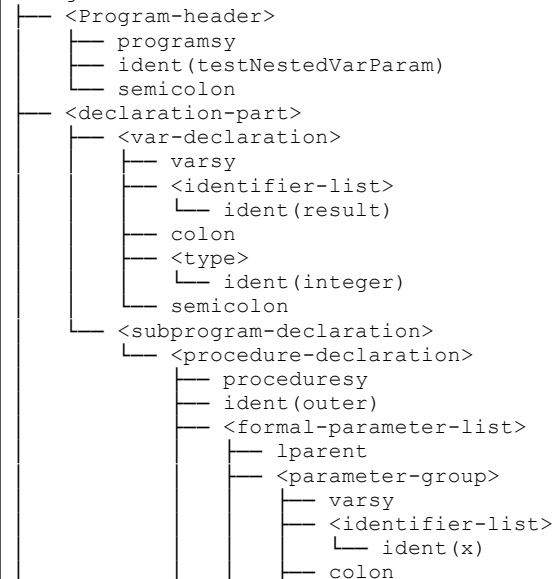
semicolon
proceduresy
ident(inner)
semicolon
beginsy
ident(x)
becomes
ident(x)
times
intcon(2)
endsy
semicolon
beginsy
ident(x)
becomes
ident(x)
plus
intcon(5)
semicolon
ident(inner)
endsy
semicolon
beginsy
ident(result)
becomes
intcon(10)
semicolon
ident(writeln)
lparent
ident(result)
rparent
semicolon
ident(outer)
lparent
ident(result)
rparent
semicolon
ident(writeln)
lparent
ident(result)
rparent
endsy
period

```

Syntax error: unexpected token endsy, expected lparent

Syntax error: unexpected token endsy, expected rparent

<Program>





```

    <statement>
    └─ <assignment-statement>
        └─ <variable>
            └─ ident(result)
        └─ becomes
        └─ <expression>
            └─ <simple-expression>
                └─ <term>
                    └─ <factor>
                        └─ intcon(10)

    semicolon
    <statement>
    └─ <procedure-function-call>
        └─ ident(writeln)
        └─ lparent
        └─ <parameter-list>
            └─ <expression>
                └─ <simple-expression>
                    └─ <term>
                        └─ <factor>
                            └─ <variable>
                                └─ ident(result)
        └─ rparent

    semicolon
    <statement>
    └─ <procedure-function-call>
        └─ ident(outer)
        └─ lparent
        └─ <parameter-list>
            └─ <expression>
                └─ <simple-expression>
                    └─ <term>
                        └─ <factor>
                            └─ <variable>
                                └─ ident(result)
        └─ rparent

    semicolon
    <statement>
    └─ <procedure-function-call>
        └─ ident(writeln)
        └─ lparent
        └─ <parameter-list>
            └─ <expression>
                └─ <simple-expression>
                    └─ <term>
                        └─ <factor>
                            └─ <variable>
                                └─ ident(result)
        └─ rparent

    endsy
period

```

===== Decorated AST =====

Program: testNestedVarParam [tab=43] type=void lev=0 line=1:1

└─ Declarations: [2 declarations] line=2:1

└─ VarDecl: result [tab=44] type=integer lev=1 line=2:13 <type simple=integer resolved=integer>

└─ Procedure: outer [tab=45] type=void lev=1 line=4:1

└─ Params:

└─ VarDecl: x [tab=46] type=integer lev=2 line=4:17 <type simple=integer resolved=integer>

└─ Locals:

└─ VarDecl: temp [tab=47] type=integer lev=2 line=5:11 <type simple=integer resolved=integer>

└─ Procedure: inner [tab=48] type=void lev=2 line=7:5

└─ Locals:

└─ Block: [1 statements] line=9:9

└─ Assign: type=integer line=9:9

└─ Var: x [tab=46] type=integer lev=2 lval=true line=9:9

└─ BinOp: * type=integer line=9:16

```

    Var: x [tab=46] type=integer lev=2 lval=true line=9:14
    Number: 2 type=integer line=9:18
  Block: [2 statements] line=13:5
    Assign: type=integer line=13:5
      Var: x [tab=46] type=integer lev=2 lval=true line=13:5
      BinOp: + type=integer line=13:12
        Var: x [tab=46] type=integer lev=2 lval=true line=13:10
        Number: 5 type=integer line=13:14
      Call: inner [tab=48] type=void lev=2 args=0 line=14:5
Block: [4 statements] line=18:5
  Assign: type=integer line=18:5
    Var: result [tab=44] type=integer lev=1 lval=true line=18:5
    Number: 10 type=integer line=18:15
  Call: writeln [tab=39] type=void lev=0 args=1 line=19:5
    Var: result [tab=44] type=integer lev=1 lval=true line=19:13
  Call: outer [tab=45] type=void lev=1 args=1 line=20:5
    Var: result [tab=44] type=integer lev=1 lval=true line=20:11
  Call: writeln [tab=39] type=void lev=0 args=1 line=21:5
    Var: result [tab=44] type=integer lev=1 lval=true line=21:13

```

=== Symbol Table (tab) ===

idx	id	obj	type	ref	nrm	lev	adr	link
0	and	keyword	void	0	value	0	0	-
1	array	keyword	void	0	value	0	0	-
2	begin	keyword	void	0	value	0	0	-
3	case	keyword	void	0	value	0	0	-
4	const	keyword	void	0	value	0	0	-
5	div	keyword	void	0	value	0	0	-
6	downto	keyword	void	0	value	0	0	-
7	do	keyword	void	0	value	0	0	-
8	else	keyword	void	0	value	0	0	-
9	end	keyword	void	0	value	0	0	-
10	for	keyword	void	0	value	0	0	-
11	function	keyword	void	0	value	0	0	-
12	if	keyword	void	0	value	0	0	-
13	mod	keyword	void	0	value	0	0	-
14	not	keyword	void	0	value	0	0	-
15	of	keyword	void	0	value	0	0	-
16	or	keyword	void	0	value	0	0	-
17	procedure	keyword	void	0	value	0	0	-
18	program	keyword	void	0	value	0	0	-
19	record	keyword	void	0	value	0	0	-
20	repeat	keyword	void	0	value	0	0	-
21	integer	keyword	void	0	value	0	0	-
22	real	keyword	void	0	value	0	0	-
23	boolean	keyword	void	0	value	0	0	-
24	char	keyword	void	0	value	0	0	-
25	string	keyword	void	0	value	0	0	-
26	then	keyword	void	0	value	0	0	-
27	to	keyword	void	0	value	0	0	-
28	type	keyword	void	0	value	0	0	-
29	until	keyword	void	0	value	0	0	-
30	var	keyword	void	0	value	0	0	-
31	while	keyword	void	0	value	0	0	-
32	integer	type	integer	0	value	0	1	-
33	real	type	real	0	value	0	1	32
34	boolean	type	boolean	0	value	0	1	33
35	char	type	char	0	value	0	1	34
36	string	type	string	0	value	0	1	35
37	true	constant	boolean	0	value	0	1	36
38	false	constant	boolean	0	value	0	0	37
39	writeln	procedure	void	0	value	0	0	38
40	readln	procedure	void	0	value	0	0	39
41	write	procedure	void	0	value	0	0	40
42	read	procedure	void	0	value	0	0	41
43	testNestedVarParamprogram	void	1	value	0	0	0	42
44	result	variable	integer	0	value	1	0	-
45	outer	procedure	void	2	value	1	0	44
46	x	variable	integer	0	ref	2	0	-

47	temp	variable	integer	0	value	2	1	46
48	inner	procedure	void	3	value	2	0	47

=== Block Table (btab) ===

idx	last	lpar	psze	vsze
0	43	0	0	0
1	45	-	0	1
2	48	46	1	1
3	-	-	0	0

=== Array Table (atab) ===

idx	xtp	etyp	eref	low	high	elsz	size
(empty)							

===== Intermediate Code =====

```

1 JMP 0 20 ; skip subprogram bodies
2 INT 0 4 ; outer frame (4 slots)
3 STO 0 3 ; param x
4 JMP 0 12 ; skip nested subprograms
5 INT 0 3 ; inner frame (3 slots)
6 LOD 1 3 ; load x addr
7 LDI 0 0 ; load x val (ref)
8 LIT 0 2 ; int 2
9 OPR 0 4 ; op *
10 LOD 1 3 ; load x addr
11 STI 0 0 ; store x (ref)
12 RET 0 0 ; return from inner
13 LOD 0 3 ; load x addr
14 LDI 0 0 ; load x val (ref)
15 LIT 0 5 ; int 5
16 OPR 0 2 ; op +
17 LOD 0 3 ; load x addr
18 STI 0 0 ; store x (ref)
19 CAL 0 4 ; call inner
20 RET 0 0 ; return from outer
21 INT 0 4 ; main program frame (4 slots)
22 LIT 0 10 ; int 10
23 STO 0 3 ; store result
24 LOD 0 3 ; load result
25 OPR 0 14 ; writeln arg
26 LDA 0 3 ; addr result
27 CAL 0 1 ; call outer
28 LOD 0 3 ; load result
29 OPR 0 14 ; writeln arg
30 RET 0 0 ; program end

```

===== Runtime Output =====

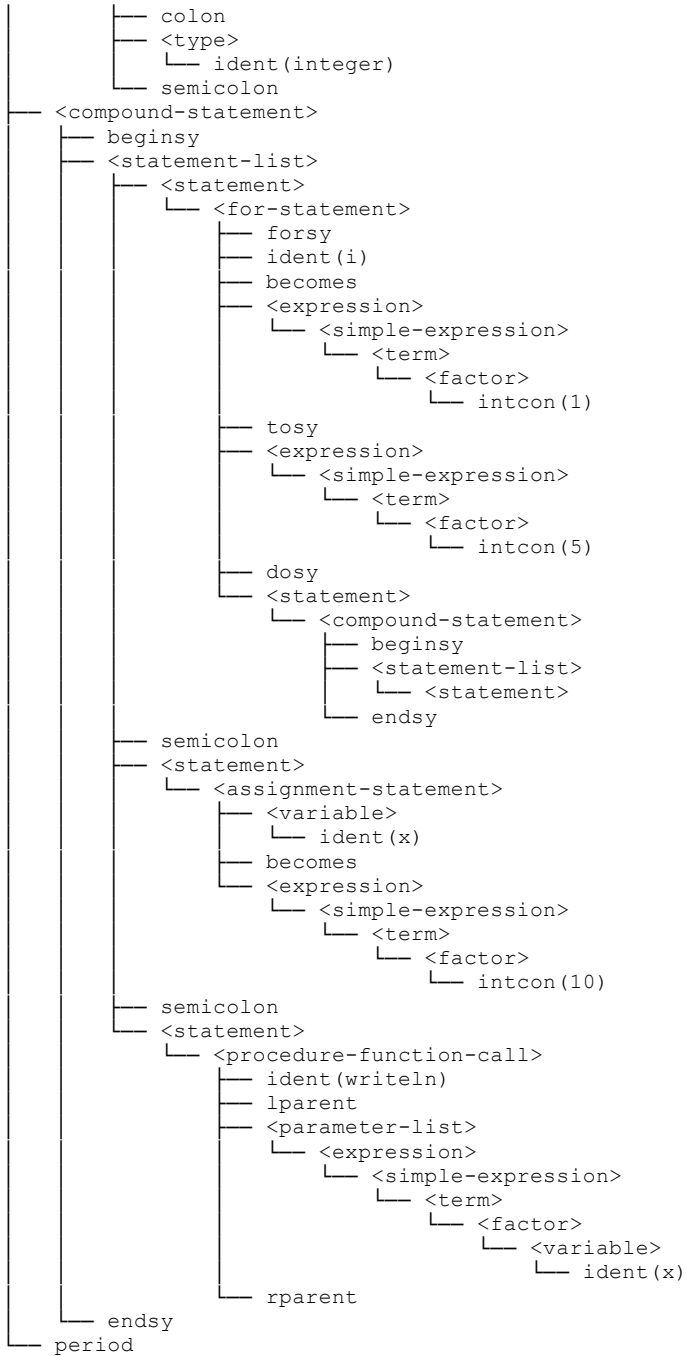
```

10
30

```

3.5 Pengujian 5: input-18.txt

Masukan (input)
<pre>program testEmptyCompoundFor; var i, x: integer; begin for i := 1 to 5 do begin end; x := 10; writeln(x) end.</pre>
Keluaran (output)
<pre>programsy ident(testEmptyCompoundFor) semicolon varsy ident(i) comma ident(x) colon ident(integer) semicolon beginsy forsy ident(i) becomes intcon(1) tosy intcon(5) dosy beginsy endsy semicolon ident(x) becomes intcon(10) semicolon ident(writeln) lparent ident(x) rparent endsy period <Program> ├── <Program-header> │ ├── programsy │ ├── ident(testEmptyCompoundFor) │ └── semicolon ├── <declaration-part> │ └── <var-declaration> │ ├── varsy │ └── <identifier-list> │ ├── ident(i) │ ├── comma │ └── ident(x)</pre>



===== Decorated AST =====

Program: testEmptyCompoundFor [tab=43] type=void lev=0 line=1:1

└─ Declarations: [2 declarations] line=2:1

└─ VarDecl: i [tab=44] type=integer lev=1 line=2:11 <type simple=integer resolved=integer>

└─ VarDecl: x [tab=45] type=integer lev=1 line=2:11 <type simple=integer resolved=integer>

└─ Block: [3 statements] line=4:5

└─ For: i to [varTab=44] type=void line=4:5

└─ Number: 1 type=integer line=4:14

└─ Number: 5 type=integer line=4:19

└─ Block: [1 statements]

└─ EmptyStmt

=== Symbol Table (tab) ===

```
=== Block Table (btab) ===
```

=== Array Table (atab) ===

```
===== Intermediate Code =====
```

IF2224 TBFO - 40

```
3 LIT 0 1 ; int 1
4 STO 0 3 ; for: init
5 LOD 0 3 ; for: load var
6 LIT 0 5 ; int 5
7 OPR 0 12 ; for: check <=
8 JPC 0 13 ; for: exit
9 LOD 0 3 ; for: load var for inc
10 LIT 0 1 ; for: 1
11 OPR 0 2 ; for: inc/dec
12 STO 0 3 ; for: store var
13 JMP 0 4 ; for: back to check
14 LIT 0 10 ; int 10
15 STO 0 4 ; store x
16 LOD 0 4 ; load x
17 OPR 0 14 ; writeln arg
18 RET 0 0 ; program end

===== Runtime Output =====
10
```

BAB 4 KESIMPULAN DAN SARAN

4.1 Kesimpulan

Pada Milestone 4, program telah berhasil mengimplementasikan tahap intermediate code generation dan interpreter. Intermediate Code Generator menerima Decorated AST serta Symbol Table dari tahap semantic analysis, lalu mengubahnya menjadi daftar instruksi linear. Instruksi tersebut mencakup operasi dasar seperti LIT, LOD, STO, CAL, INT, JMP, JPC, OPR, dan RET, serta instruksi tambahan untuk mendukung akses tidak langsung.

Interpreter kemudian menjalankan intermediate code menggunakan pendekatan stack machine. Stack machine menyimpan frame, variabel, parameter, dan operand sementara selama eksekusi program. Dengan mekanisme ini, program dapat mengeksekusi assignment, ekspresi aritmatika dan relasional, percabangan, perulangan, pemanggilan prosedur/fungsi, serta output melalui write dan writeln.

Secara umum, implementasi Milestone 4 sudah memenuhi struktur utama yang diminta, yaitu adanya intermediate code, stack penyimpanan, dan interpreter. Namun, masih terdapat beberapa bagian yang perlu diperbaiki agar hasil eksekusi semakin sesuai dengan spesifikasi, terutama terkait konsistensi tipe runtime, validasi error, dan penanganan beberapa konstruksi bahasa.

4.2 Saran

- Memperkaya pesan error dengan source-line context selain nomor baris, mengikuti praktik error reporting compiler modern seperti Rustc atau Clang.
- Menambahkan pengujian yang lebih banyak, terutama untuk kasus yang belum umum, seperti case, char, function return, dan error runtime.
- Merapikan pelaporan error dan intermediate code agar lebih mudah dibaca, ditelusuri, dan dinilai saat debugging maupun demo.

LAMPIRAN

Link Repository *Github* : <https://github.com/road2qnt/ABS-Tubes-IF2224-2026>

Pembagian Tugas

Nama	Pembagian Tugas	Kontribusi
Ega Luthfi Rais	Implementasi Code, Debugging, dan Testing	33.3%
Salman Faiz Assidqi	-	0%
Ahmad Fauzan Putra	Laporan, Testing	33.3%
Leonardus Brain Fatosja	Inisiasi Awal, Laporan, Debugging	33.3%

REFERENSI

- GeeksforGeeks. (2026). Three address code in Compiler. Diakses dari <https://www.geeksforgeeks.org/compiler-design/three-address-code-compiler/>
- GeeksforGeeks. (2026). Intermediate Code Generation in Compiler Design Diakses dari <https://www.geeksforgeeks.org/compiler-design/intermediate-code-generation-in-compiler-design/>
- GeeksforGeeks. (2025). Introduction to Interpreters. Diakses dari <https://www.geeksforgeeks.org/compiler-design/introduction-to-interpreters/>